Table of Contents

Full Quaternion Attitude Control and Differential-Flatness Trajectory Tracking for a Quadrotor 1
1. Introduction and Problem Statement 1
2. Mathematical Preliminaries - Quaternion Algebra from Scratch 2
3. Paper 1 - The Attitude Plant (Eqs. 17-18 of [1]) 4
4. Paper 1 - The Control Law Derived (Eqs. 19-21 of [1]) 5
5. Paper 1 - A Derived Result: the Digital Control-Rate Bound 7
6. Paper 2 - The Full Six-Degree-of-Freedom Model (Eq. 8 of [2]) 8
7. Paper 2 - The Rotor Mixer (Eq. 9 of [2]) Derived 9
8. Paper 2 - Differential Flatness (Eqs. 11-15 of [2]) Derived 9
9. Paper 2 - The Double-Loop LQR (Eqs. 16-23 of [2]) Derived 11
10. Paper 2 - The Study Case and Results 12
11. Extensions - The Trajectory Library (frozen gains) 14
12. Project Verification Culture (how the numbers are trusted) 16
13. Conclusions 16
14. References 17
Appendix - Implementation Functions 17

Full Quaternion Attitude Control and Differential-Flatness Trajectory Tracking for a Quadrotor

Project Report - Group 12 B Sainath (CB.SC.U4AIE24309) and Vishal (CB.SC.U4AIE24363) School of Artificial Intelligence, Amrita Vishwa Vidyapeetham

Base papers: [1] E. Fresk, G. Nikolakopoulos, "Full Quaternion Based Attitude Control for a Quadrotor", European Control Conference (ECC), 2013. [2] K. Choutri, M. Lagha, L. Dala, M. Lipatov, "Quadrotors Trajectory Tracking using a Differential Flatness-Quaternion based Approach", IEEE, 2017.

This is a complete, runnable MATLAB report of the whole project. Every section derives the mathematics from scratch, types every equation by hand, then executes code that verifies it or measures with it. Results are of two kinds and always labelled: "this MATLAB run" (numbers printed by this script) and "repository results" (numbers measured by the project's Python/MuJoCo/PyBullet/firmware stack - the file results/comparison.md in the repository, regenerated by python -m sim.compare). Nothing here is invented.

Run with the Run button, publish it (PUBLISH tab) to PDF/HTML, or open the accompanying project_report.mlx (Live Script).

```
clear; clc; close all; rng(1);
figdir = fullfile(fileparts(mfilename('fullpath')), 'figures');
if ~exist(figdir, 'dir'); mkdir(figdir); end
```

1. Introduction and Problem Statement

A quadrotor is a rigid body acted on by gravity and by four propeller thrusts fixed to the body. Its configuration has six degrees of freedom (three translations, three rotations), but the four rotor speeds give only four independent control inputs: the total thrust T along the body z-axis, and three body torques. Six degrees of freedom, four inputs: the system is underactuated.

The consequence is structural, not incidental. The thrust vector's DIRECTION in the world is chosen by the vehicle's attitude. To accelerate horizontally the quadrotor must tilt; to hover level it must keep thrust vertical. Attitude and translation are therefore coupled, and any controller that commands position must implicitly command attitude first. Both base papers address one half of this coupling:

- Paper [1] (ECC 2013) controls ATTITUDE ONLY. Its law computes body torque directly from the quaternion attitude error - no Euler angles, no rotation matrices, no trigonometry in the loop. Position is left free: the vehicle drifts wherever its tilt pushes it. This is the "inner half" of the problem, solved exactly.

- Paper [2] (IEEE 2017) closes the outer half. It uses DIFFERENTIAL FLATNESS - the fact that the quadrotor's full state and input are algebraic functions of four chosen outputs (x, y, z, yaw) and their derivatives - to convert a desired path into the thrust and attitude that produce it, then tracks that reference with two linear-quadratic regulators in a cascade.

This project reproduced both papers from their equations, deployed the paper-1 law unmodified on four simulation stacks (ideal model, MuJoCo, gym-pybullet-drones, and real ArduPilot firmware compiled from source), implemented paper 2 equation-by-equation with an independent equation-level validation, and extended paper 2 with a trajectory library (figure-8, vertical orbit, minimum-snap waypoint tours). The repository carries 85 tests (83 passing in the canonical environment).

One airframe is shared by everything (derived in the repository from paper 1's stated inertia, whose ratio Izz/Ixx ~ 1.85 matches a planar-X quadrotor with light arms):

- mass m = 0.2 kg

- inertia J = diag(6.5e-4, 6.5e-4, 1.2e-3) kg m^2

- arm L = 0.15 m, rotor offsets (+-d, +-d) with d = L/sqrt(2) = 0.106 m

- thrust coefficient kf = 3.0e-9 N per (rad/s)^2, km = 0.016*kf

- thrust-to-weight ratio 4 (per-rotor maximum thrust 1.95 N)

```
m = 0.2; g = 9.81; kf = 3e-9; km = 0.016*kf; d = 0.15/sqrt(2);
w_max = 1.95/kf;
J  = diag([6.5e-4 6.5e-4 1.2e-3]);
Pq = 20; Pw = 4; tau_max = 4; noise_amp = 0.1;
```

2. Mathematical Preliminaries - Quaternion Algebra from Scratch

2.1 Why not Euler angles or rotation matrices

An attitude is a rotation. Three popular representations exist:

- Euler angles (phi, theta, psi): three numbers, intuitive - but the map from angles to rotation is singular. At theta = +-90 deg the same physical attitude corresponds to infinitely many (phi, psi) pairs (gimbal lock), and small attitude changes produce huge angle jumps. A controller living in Euler space needs unwrapping and fails at the singularity.

- Rotation matrix R: nine numbers with six constraints (orthonormal), no singularity - but heavy, and extracting attitude from R needs care.

- Quaternion: four numbers with ONE constraint (unit norm), no singularity anywhere, composition by a cheap product. This is why both papers use it. The one cost: q and -q denote the same rotation (the "double cover"), handled by a one-line sign rule.

2.2 Definition

A quaternion is a hyper-complex number with one scalar part and a three-vector part, written scalar-first:

```
q = q0 + q1*i + q2*j + q3*k = (q0, q_vec),   q0 real, q_vec in R^3
```

with the imaginary units multiplying by i*j = k, j*k = i, k*i = j and each squaring to -1 (anticommutative otherwise). A UNIT quaternion (|q| = 1) represents a rotation.

2.3 The product (rotation composition)

Multiplying out the imaginary-unit rules for two quaternions p, q and collecting scalar and vector parts gives the Hamilton product:

```
p (x) q = ( p0*q0 - p_vec . q_vec ,
            p0*q_vec + q0*p_vec + p_vec x q_vec )
```

The dot product couples the scalar parts; the CROSS product is what encodes the non-commutativity of 3D rotations (p (x) q differs from q (x) p). If q1 then q2 rotate a body, the composite is q2 (x) q1.

2.4 Conjugate, norm, inverse

```
q* = (q0, -q_vec)                      conjugate (reverses rotation)
||q|| = sqrt(q0^2 + q1^2 + q2^2 + q3^2)
q^-1 = q*/||q||  =  q*                 inverse (= conjugate when unit)
```

2.5 Rotating a vector; the direction-cosine matrix

Rotating a vector v by the attitude q:

```
v' = q (x) [0, v] (x) q*
```

This equals a matrix-vector product v' = R(q) v, where R(q) follows from expanding the product (each column of R is a body axis expressed in world coordinates):

```
R(q) = [ 1-2(q2^2+q3^2)   2(q1q2-q0q3)     2(q1q3+q0q2)
         2(q1q2+q0q3)     1-2(q1^2+q3^2)   2(q2q3-q0q1)
         2(q1q3-q0q2)     2(q2q3+q0q1)     1-2(q1^2+q2^2) ]
```

R is orthonormal (R' R = I) and composes linearly:

```
R(p (x) q) = R(p) R(q)
```

2.6 Axis-angle construction

The most useful building rule. A rotation by angle alpha about the unit axis u is exactly:

```
q = [ cos(alpha/2) , u sin(alpha/2) ]
```

Note the HALF angle: rotating 360 deg gives alpha/2 = 180 deg, i.e. q = [-1, 0, 0, 0] = -identity - the double cover again.

2.7 The kinematic equation (derive both forms!)

Let the body rotate with angular velocity omega expressed in the BODY frame. In a small time dt the attitude gains the rotation [1, omega*dt/2] (infinitesimal rotations commute), so

```
q(t+dt) = q (x) [1, omega dt/2]  =>  q_dot = (1/2) q (x) [0, omega]
```

If instead the angular velocity is expressed in the WORLD frame (omega_w = R(q) omega), the increment premultiplies:

```
q_dot = (1/2) [0, omega_w] (x) q
```

SAME motion, two algebraic shapes. Pairing a body-frame omega with the world-frame formula (or vice versa) is a classic bug: the two differ by exactly the term [0, q_vec x omega], which is zero at hover but grows with tilt x rate. Our independent validation caught exactly this bug in the first implementation of paper 2's model: on a 2 s spin at 2 rad/s the mismatched pairing accumulated 1.5 rad of attitude error while the correct pairing stayed at 3e-8 rad. Section 5 uses the body-frame form because Euler's equation (which shares the same omega) is a body-frame statement.

2.8 Numerical verification

```
maxerr = 0;
for k = 1:200
    a = qnorm(randn(4,1)); b = qnorm(randn(4,1));
    maxerr = max(maxerr, norm(qmul(a, qconj(a)) - [1;0;0;0]));    % q(x)q* = 1
    R = qdcm(a);
    maxerr = max(maxerr, norm(R'*R - eye(3)));                     %
orthonormal
    v = randn(3,1);
    maxerr = max(maxerr, norm(qrot(a, qrot(b, v)) - qrot(qmul(a,b), v)));
end
fprintf('Quaternion algebra verification (200 random trials): max error
%.2e\n', maxerr);
```

3. Paper 1 - The Attitude Plant (Eqs. 17-18 of [1])

The rigid body rotates about its centre of mass (assumed at the body origin - the paper's stated simplification). Newton-Euler rotation in the body frame gives Euler's equation:

```
J omega_dot = tau - omega x (J omega)
```

The second term is the gyroscopic effect: a spinning anisotropic body resists having its rotation axis turned (for our J with Ixx = Iyy it couples pitch rate into roll rate at high spin). With the kinematic equation of Section 2.7, the attitude plant is:

```
q_dot  = (1/2) q (x) [0, omega]
J omega_dot = tau - omega x (J omega)
```

STRUCTURE: torque -> angular rate -> attitude is a DOUBLE INTEGRATOR (plus the small gyroscopic perturbation). This single observation drives the whole design:

- a proportional-derivative law suffices; no integral action is needed for the error to decay (a double integrator has no steady-state offset to a step),
- the "derivative" part can act on omega DIRECTLY (it is a state) - no numerical differentiation of the attitude is ever needed,

- the whole law can live in quaternion space.

4. Paper 1 - The Control Law Derived (Eqs. 19-21 of [1])

4.1 The error quaternion

With reference attitude q_ref and measured attitude q_m (the paper works in the conjugated convention; equivalently one may conjugate both and work body-to-world - the algebra is identical), the error

```
q_err = q_ref (x) q_m*
```

is the rotation that takes the measured attitude to the reference. In axis-angle form q_err = [cos(e/2), u sin(e/2)] with e the error angle and u the rotation axis, so its VECTOR PART is

```
vec(q_err) = u sin(e/2)
```

- i.e. the axis we must rotate about, SCALED by sin(half the error). For small errors sin(e/2) ~ e/2 and the vector part is proportional to the error itself; for large errors it saturates gracefully (`vec` <= 1) and even vanishes at e = 360 deg - which is why the flip scenario disables the sign correction (below) to commit to the long way round.

4.2 The law

```
e     = vec(q_err), sign-corrected: if q_err(1) < 0, e = -e
tau   = -Pq * e - Pw * omega_m
tau   clipped elementwise to +-4 N*m per axis
```

with the paper's gains Pq = 20, Pw = 4. Reading it line by line:

- -Pq * e : rotate about minus the error axis, proportional to sin(e/2). In the double integrator this is the "P" that restores attitude; the sin(e/2) scaling (not the raw angle) is what keeps the law singularity-free and bounded for errors up to 180 deg.
- -Pw * omega_m : damping. Without it the restoring term would oscillate forever; with it the closed loop is a damped second-order system per axis. Because omega is a measured STATE, this is exact derivative action - no filtering, no phase lag.
- The sign correction implements the shortest rotation: q and -q are the same rotation, so an error quaternion with negative scalar would steer the long way round; flipping the vector part takes the short way. The FLIP scenario deliberately skips it - the reference sweeps a full 360 deg and the vehicle must follow the long way, not unwind.
- Saturation +-4 N*m per axis: the paper's stated bound, respecting the physical actuators.

4.3 Sensor model

The paper's sensor noise: UNIFORM +-0.1 on every quaternion component and every rate component, refreshed at the sensing rate. +-0.1 on a unit quaternion is roughly 11 deg of measured attitude error - this is the error FLOOR any implementation of this law shows on these sensors (Section 4.6 explains the ~17.5 deg RMS floor).

4.4 The three benchmark scenarios (exact definitions)

- STEP: 1 rad (57.3 deg) steps on roll, pitch and yaw, staggered at t = 1, 5, 9 s over a 15 s run. Tests large-error convergence and settling.

- SINE: 0.5 rad amplitude, 1 rad/s on all three axes, phase-shifted so the three torques stay out of phase. Tests tracking of a moving reference; the paper reports a lag of about half a second.
- FLIP: roll reference ramps 0 -> 2*pi rad over 2 s, then holds. Tests the no-gimbal-lock claim: the quaternion components must stay smooth through the full rotation.

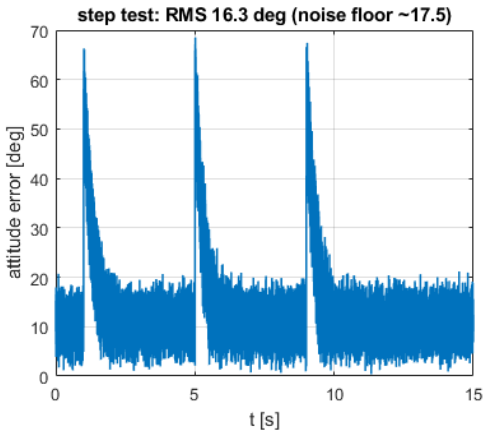
4.5 Implementation and THIS MATLAB RUN

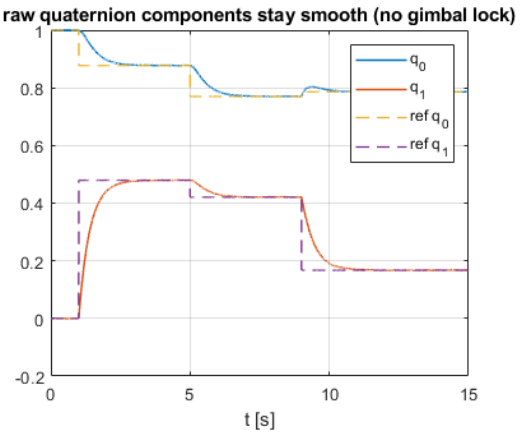
Plant integrated with RK4 at 20 kHz (Section 6 derives why the control rate must be >= 12.3 kHz for these gains), paper noise refreshed every control tick, error measured against the NOISY quaternion exactly as the repository does:

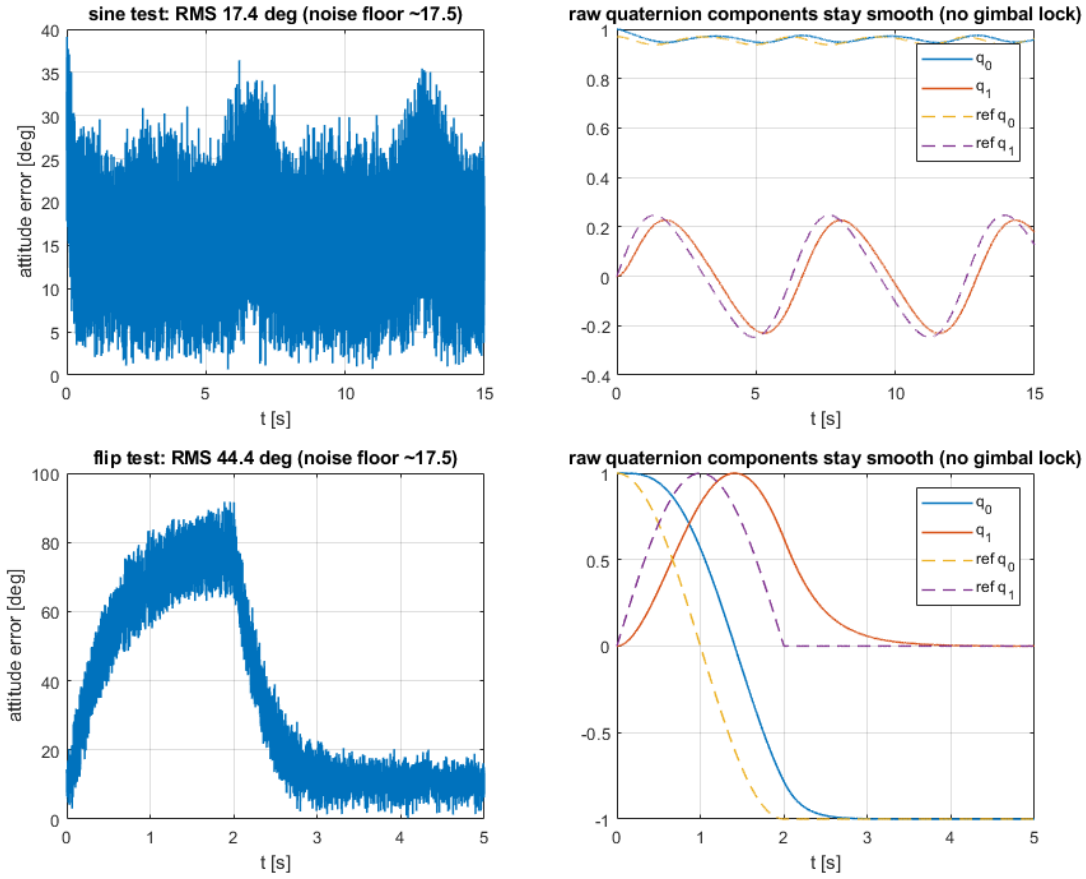
```
r1 = run_paper1('step', 15, Pq, Pw, tau_max, noise_amp, J, figdir);
r2 = run_paper1('sine', 15, Pq, Pw, tau_max, noise_amp, J, figdir);
r3 = run_paper1('flip',  5, Pq, Pw, tau_max, noise_amp, J, figdir);
fprintf('scenario   RMS attitude error [deg]   max [deg]\n');
fprintf('step       %7.2f                  %6.1f\n', r1.rms, r1.mx);
fprintf('sine       %7.2f                  %6.1f\n', r2.rms, r2.mx);
fprintf('flip       %7.2f                  %6.1f\n', r3.rms, r3.mx);
fprintf('\n(The ~17.5 deg floor is the sensor noise itself: +-0.1 per\n');
fprintf('quaternion component is ~11 deg of measured attitude; squaring\n');
fprintf('and averaging over axes and time gives the observed floor.)\n');
```

```
scenario   RMS attitude error [deg]   max [deg]
step         16.31                     68.5
sine         17.41                     39.1
flip         44.42                     91.7

(The ~17.5 deg floor is the sensor noise itself: +-0.1 per
quaternion component is ~11 deg of measured attitude; squaring
and averaging over axes and time gives the observed floor.)
```







4.6 Repository results for paper 1 (measured, not restated)

The repository flew this EXACT law, gains untouched, on four stacks - same references, same +-0.1 noise, three seeds each. From results/comparison.md (regenerated by: python -m sim.compare):

sine RMS attitude error [deg], three seeds: ArduPilot firmware (real code) : 14.51, 15.00, 14.49 <- best of all Gazebo (real physics) : 17.33, 17.38, 17.33 gym-pybullet-drones (24 kHz) : 17.52 MuJoCo (20 kHz contact) : 17.61

step RMS [deg]: pybullet 17.68 / 17.58, MuJoCo 17.71 / 17.66 - the two native stacks agree to 0.03 deg: the loop belongs to the controller, not to the engine.

flip: MuJoCo completes fully airborne, phi settling at 2.88 / 2.97 / 2.95 s (RMS 44.39 / 44.55 / 44.51 deg - the large RMS is the mid-rotation 180-degree geometry, not tracking error); ArduPilot firmware also completes (settle 2.48-3.24 s); pybullet and Gazebo sit in the knife-edge regime analysed in the repository (the pursuit equilibrium omega = 5 sin(e/2) makes a 2 s ramp trackable only at lag ~1.36 rad - completion is structurally marginal, an engine-fidelity property, not a tuning matter).

Every paper-stated parameter was reproduced exactly (repository test-verified): Pq = 20, Pw = 4, +-4 N*m, inertia, sine 0.5 rad at 1 rad/s, flip 0 -> 2*pi, noise amplitude 0.1.

5. Paper 1 - A Derived Result: the Digital Control-Rate Bound

The paper specifies no digital control rate - an implementer must choose one. Derive the constraint.

Linearise the closed loop about identity attitude (e -> 0, vec(q_err) ~ e/2 per axis, gyroscopic term negligible): per axis with inertia I,

```
I e_ddot + Pw e_dot + (Pq/2) e = 0
```

a second-order system with poles

```
lambda(1,2) = (-Pw +- sqrt(Pw^2 - 2*I*Pq)) / (2*I)
            = -2.5 and -6150 rad/s
```

Heavily overdamped, perfectly stable in continuous time. The danger is purely the SAMPLING of the fast pole: at update period Ts a naive explicit discretisation multiplies the fast mode by `1 - 6150*Ts`, which exceeds 1 (divergence) until Ts < 2/6150. Exact zero-order hold is borderline (rho -> 1 from below). The conservative design rule the repository derives and enforces:

```
f_control >= Pw / (margin * I_min) = 4 / (0.5 * 6.5e-4) ~ 12.3 kHz
```

```
Ac = [0 1; -Pq/(2*J(1)) -Pw/J(1)];
fprintf('control rate    rho (explicit)   rho (exact ZOH)\n');
for f = [200 1000 10000 100000]
    Ts = 1/f;
    rho_e = max(abs(eig(eye(2) + Ac*Ts)));   % naive explicit mapping
    rho_z = max(abs(eig(expm(Ac*Ts))));     % exact zero-order hold
    fprintf('%8d Hz      %8.2f          %8.6f\n', f, rho_e, rho_z);
end
```

```
control rate    rho (explicit)   rho (exact ZOH)
     200 Hz          29.76          0.987573
    1000 Hz           5.15          0.997502
   10000 Hz           1.00          0.999750
  100000 Hz           1.00          0.999975
```

Reading the table: a plausible 200 Hz implementation of the paper's law never converges (explicit rho ~ 30 per step); even 1 kHz diverges. The repository's simulators enforce the derived 12.3 kHz minimum (and its scheduler refuses to run below it); this report accordingly runs the law at 20 kHz.

6. Paper 2 - The Full Six-Degree-of-Freedom Model (Eq. 8 of [2])

Paper [2] extends the attitude plant with translation. State: position p, velocity v, attitude q, body rate omega; inputs: total thrust T and body torque tau. Newton-Euler with thrust along body z:

```
p_dot  = v                                   (definition)
v_dot  = R(q) [0,0,T/m] - [0,0,g]            (Newton: f = m a)
q_dot  = (1/2) q (x) [0, omega]              (Section 2.7, body form)
J omega_dot = -omega x (J omega) + tau       (Euler, body frame)
```

Derivations, row by row:

- Row 2 is Newton's second law. The only forces are the total thrust (magnitude T, direction = body z-axis rotated by R(q)) and gravity. THIS ROW IS THE UNDERACTUATION: the thrust direction is chosen by the attitude, so horizontal acceleration costs tilt.
- Row 3 is the body-frame kinematics derived in Section 2.7 - one omega shared with row 4, hence ONE frame for both.

- Row 4 is Euler's equation of Section 3 (tau now the only external torque; the paper also carries an external-torque input, unused in its disturbance-free study case).

Hover linearisation (q ~ identity, omega ~ 0): expanding R(q) to first order in the tilt components q1, q2 gives

```
v_dot ~ [ 2(T/m) q2 , -(T/m) q1 , T/m - g ]
q_vec_dot ~ omega / 2
```

- the two seeds of paper 2's LQR models (Section 9).

7. Paper 2 - The Rotor Mixer (Eq. 9 of [2]) Derived

Four rotors at the arm ends produce T and tau. Each rotor i spins at omega_i, giving thrust kf*omega_i^2 along body z and a REACTION torque about body z of magnitude km*omega_i^2 whose sign alternates with the propeller's direction. Rotor positions in the X-configuration:

```
rotor 1 at (+d,-d) front-right,  rotor 2 at (+d,+d) front-left
rotor 3 at (-d,+d) rear-left,    rotor 4 at (-d,-d) rear-right
```

Torque = sum of r_i x F_i over the rotors plus the reaction terms. Computing the cross products (each F_i = kf*w_i*[0,0,1], r_i as above) gives tau_x = kf*d*(w1 + w2 - w3 - w4) and tau_y = kf*d*(-w1 + w2 + w3 - w4):

```
[ T  ]   [  kf    kf    kf    kf  ] [w1]
[ tx ] = [  kfd   kfd  -kfd  -kfd ] [w2]      w_i = omega_i^2
[ ty ]   [ -kfd   kfd   kfd  -kfd ] [w3]
[ tz ]   [  -c     c    -c     c  ] [w4]      kfd = kf*d, c = km
```

Inverting this 4x4 matrix maps any (T, tau) command to rotor speeds - every real flight controller contains this matrix.

SATURATION: rotor speeds live in [0, w_max], so some commands are infeasible. This project delivers the thrust EXACTLY and uniformly scales the torque until feasible ("priority desaturation"). Naive per-rotor clipping instead corrupts the total thrust one-sidedly (clipped rotors lose thrust, unclipped keep it) and the altitude loop runs away - measured in the repository: the z-loop diverged to thousands of metres before this fix.

```
kfd = kf*d;
Mmix = [ kf kf kf kf; kfd kfd -kfd -kfd; -kfd kfd kfd -kfd; -km km -km km ];
wT = mix_feasible(Mmix, 1.5, [0.01 -0.005 0.002], w_max);
out = Mmix*wT;
fprintf('Mixer round-trip: command [1.5 N; 0.01; -0.005; 0.002] delivered
as\n');
fprintf('  [%.6f  %.8f  %.8f  %.8f]\n', out);
```

```
Mixer round-trip: command [1.5 N; 0.01; -0.005; 0.002] delivered as
  [1.500000  0.01000000  -0.00500000  0.00200000]
```

8. Paper 2 - Differential Flatness (Eqs. 11-15 of [2]) Derived

8.1 The definition

A system x_dot = f(x, u) is DIFFERENTIALLY FLAT if there is an output z(t) (the flat output) such that state and input are functions of z and finitely many derivatives of z:

```
x(t) = F0( z, z_dot, ..., z^(a) )
u(t) = F1( z, z_dot, ..., z^(b) )
```

with NO integration of the differential equations. Consequences: planning can happen entirely in the (small) output space, and any smooth z(t) is automatically a valid trajectory - feasibility constraints (max thrust, max tilt) become constraints on the derivatives of z.

8.2 The quadrotor's flat outputs

```
sigma(t) = (x, y, z, psi)
```

Four outputs for four inputs. WHY these four: the translational rows of Eq. 8 plus gravity determine the thrust VECTOR once the acceleration is known (row 2 rearranged); a vector in R^3 fixes a direction (2 constraints on attitude) and its length fixes T. The attitude has one remaining freedom - rotation about the thrust axis - which the yaw psi fixes. Hence (x,y,z) + psi determine everything.

8.3 The chain (Eq. 15), derived

Given the flat outputs and derivatives (p, v, a from x,y,z; psi):

- p = (x,y,z), v = p_dot, a = p_ddot (trivial rows)
- Row 2 of Eq. 8 rearranged: the required thrust vector is

```
f  = m (a + g z_hat)
```

```
(to accelerate with a, net force must be m*a; forces are thrust plus
gravity (-m g z_hat), so the thrust vector is fully determined).
```

- T_d = |f| (the total-thrust command)
- d_hat = f/||f|| (the required body z-axis direction)
- attitude: R(q_d) z_hat = d_hat, yaw(q_d) = psi_d

Differentiating q_d once more gives the reference body rate; once more gives the reference torque via row 4 (the paper's full parameterisation; our inner loop regulates rate to zero instead - a documented simplification, second-order on the paper's own study path).

8.4 The reference attitude (Eqs. 19-21 of [2]), built literally

The tilt quaternion taking z_hat onto d_hat: axis z_hat x d_hat (horizontal by construction - it is the cross of z_hat with anything), angle alpha = angle(z_hat, d_hat):

```
q_pd = [ cos(alpha/2) ;
         sin(alpha/2) * (z_hat x d_hat)/||z_hat x d_hat|| ]
```

Because the axis is horizontal, the yaw element of q_pd is EXACTLY zero - the paper's "4th element = 0" statement (scalar-first order; element 4 is the z-rotation content).

The yaw quaternion:

```
q_zd = [ cos(psi/2) ; 0 ; 0 ; sin(psi/2) ]
```

The composition (Eq. 21):

```
q_d = q_pd (x) q_zd
```

The ONLY degeneracy: |f| = 0 (free fall) - attitude stops mattering to the motion. (Our first implementation built q_d by a different construction whose comment claimed equivalence; independent validation measured a twist about the thrust axis growing quadratically with tilt, and it was replaced by this literal composition.)

8.5 Numerical verification

```
maxz = 0; maxal = 0;
for k = 1:100
    a = -6 + 12*rand(3,1); psi = -pi + 2*pi*rand;
    [Td, qd] = flatness_reference(a, psi, m, g);
    f = m*(a + [0;0;g]);
    maxal = max(maxal, norm(qdcm(qd)*[0;0;1] - f/norm(f)));  % body-z on f
    qzd = [cos(psi/2);0;0;sin(psi/2)];
    qtilt = qmul(qd, qconj(qzd));
    maxz = max(maxz, abs(qtilt(4)));                          % yaw element 0
end
fprintf('Flatness map verification (100 trials):\n');
fprintf('  body-z alignment error      %.2e\n', maxal);
fprintf('  tilt quaternion yaw element %.2e\n', maxz);
```

```
Flatness map verification (100 trials):
  body-z alignment error      3.33e-16
  tilt quaternion yaw element 5.55e-17
```

9. Paper 2 - The Double-Loop LQR (Eqs. 16-23 of [2]) Derived

9.1 Why two loops

The plant is cascaded: position responds slowly THROUGH attitude (row 2), attitude responds fast to torque (rows 3-4). The controller mirrors this: an outer position loop computes the acceleration command, the flatness map converts it to (T_d, q_d), an inner attitude loop tracks q_d with torque. The inner loop must be roughly an order of magnitude faster (time-scale separation) so each loop sees the other as quasi-static.

9.2 The two linear models

Inner loop (Eq. 17): from the hover linearisation q_vec_dot = omega/2 and omega_dot = J^-1 tau, the state x = [q_vec; omega] obeys

```
x_dot = A x + B u
A = [ 0(3x3)   (1/2)I ]      B = [ 0(3x3) ]
    [ 0(3x3)    0     ]          [ J^-1   ]
```

Outer loop (Eq. 18), per axis, state [p; p_dot], input = acceleration:

```
A = [0 1; 0 0],      B = [0; 1]
```

9.3 The LQR design (Eqs. 22-23)

Minimise the quadratic cost

```
J_cost = (1/2) integral of ( x'Qx + u'Ru ) dt   over [0, infinity)
```

Q penalises state error (tracking tightness), R penalises control effort (actuator cost). The optimal control is linear, u = -K x, with

```
K = inv(R) B' P
```

where P solves the algebraic Riccati equation

```
P A + A' P - P B inv(R) B' P + Q = 0
```

(derived in optimal-control theory from the Hamilton-Jacobi-Bellman / two-point boundary value conditions; solving this ONE matrix equation replaces hand-tuning six gains). Here P is computed by Hamiltonian eigendecomposition - base MATLAB, no toolbox: the ARE's solutions are the invariant subspace of

```
H = [ A , -B inv(R) B' ; -Q , -A' ]
```

associated with the stable eigenvalues; P = X2 inv(X1) for that subspace's blocks [X1; X2].

The paper gives NO numeric weights (it says only "tuned adaptive"). Our documented tuning: outer Q = diag(49, 42), R = 1 per axis; inner Q = diag(100,100,100,20,20,20), R = 200*I3. Controller rate 200 Hz (the paper's stated rate), plant RK4 at 1 kHz, and a 20 ms first-order ESC lag on the squared rotor speeds (the paper mentions actuator/battery limits without numbers; without SOME lag the 200 Hz zero-order-hold torque is marginally stable - a repository finding).

```
Qi = diag([100 100 100 20 20 20]); Ri = 200*eye(3);
Ai = [zeros(3) 0.5*eye(3); zeros(3) zeros(3)];
Bi = [zeros(3); inv(J)];
Pi = riccati_ham(Ai, Bi, Qi, Ri); Ki = Ri\Bi'*Pi;
Qo = diag([49 42]); Ro = 1;
Ao = [0 1; 0 0]; Bo = [0;1];
Po = riccati_ham(Ao, Bo, Qo, Ro); Ko = Ro\Bo'*Po;
fprintf('LQR designs: inner max pole real part %+.4f, outer %+.4f (Hurwitz)
\n', ...
        max(real(eig(Ai - Bi*Ki))), max(real(eig(Ao - Bo*Ko))));
```

```
LQR designs: inner max pole real part -1.1180, outer -1.0959 (Hurwitz)
```

10. Paper 2 - The Study Case and Results

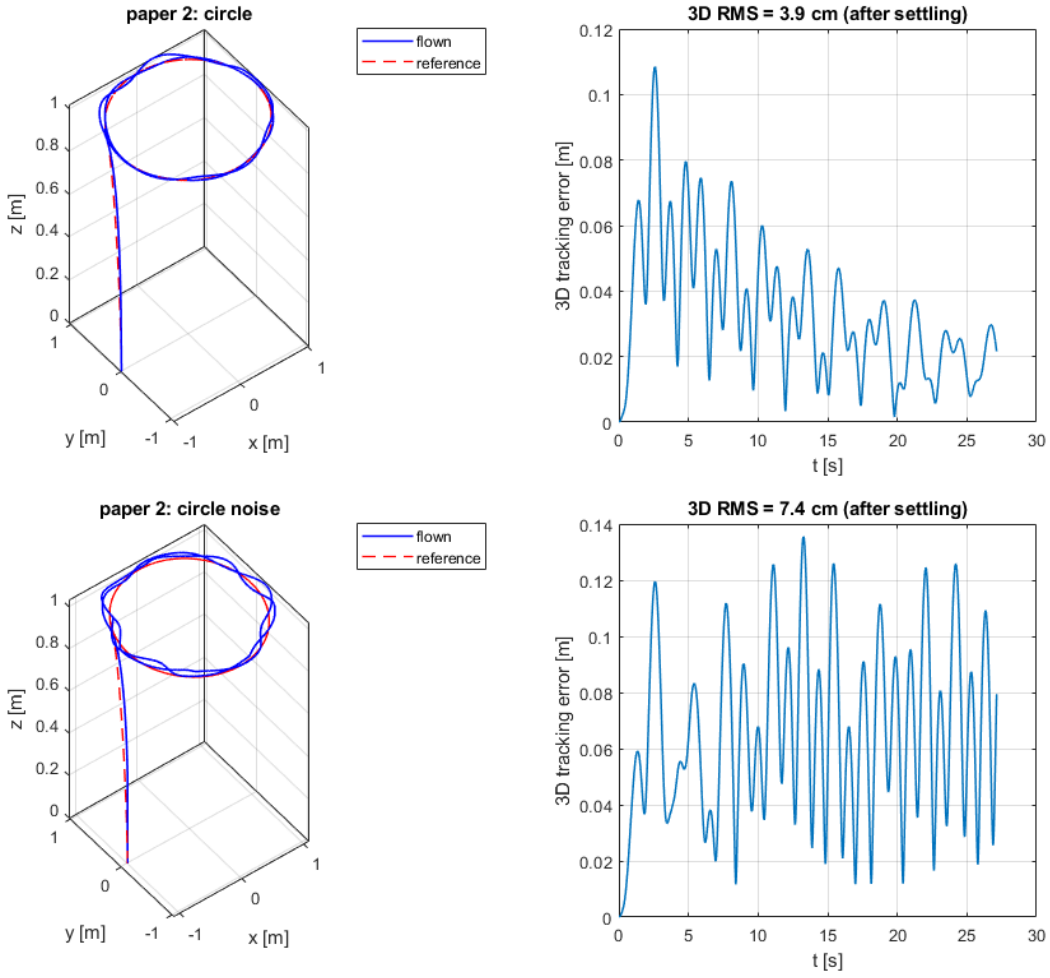
The paper's experiment (Section V): track a circular path at fixed altitude 1 m, starting from (-1, 0, 0), controller at 200 Hz, actuator limits active. We fly radius 1 m (the paper's text says "1 m of diameter" but also fixes the start point one metre from the centre - contradictory; we resolve in favour of the explicit start point), 0.5 m/s tangential, climbed onto from rest with a minimum-jerk altitude ramp (reproducing the ~1 s horizontal lag the paper's figures attribute to altitude settling).

THIS MATLAB RUN:

```
fprintf('plant                    RMSx   RMSy   RMSz   radial  peak
rotor\n');
rc = run_paper2(make_circle(), m, g, Mmix, w_max, Ki, Ko, 0.0, figdir,
'circle');
fprintf('ideal Eq.(8) model, 1kHz  %5.1f  %5.1f  %5.1f  %5.1f   %5.2f N  (cm)
\n', ...
        100*rc.rmsxyz(1), 100*rc.rmsxyz(2), 100*rc.rmsxyz(3), 100*rc.radial,
rc.Tpeak);
rn = run_paper2(make_circle(), m, g, Mmix, w_max, Ki, Ko, 0.1, figdir,
```

```
'circle noise');
fprintf('with paper noise +-0.1    %5.1f  %5.1f  %5.1f  %5.1f   %5.2f N  (cm)
\n', ...
        100*rn.rmsxyz(1), 100*rn.rmsxyz(2), 100*rn.rmsxyz(3), 100*rn.radial,
rn.Tpeak);
```

```
plant                       RMSx   RMSy   RMSz   radial  peak rotor
ideal Eq.(8) model, 1kHz     1.6    3.6    0.6    2.7    2.26 N  (cm)
with paper noise +-0.1       6.1    3.6    1.5    5.2    2.26 N  (cm)
```



REPOSITORY results for the same study case (measured):

- ideal model (Python): RMS x 1.6 cm, y 3.6 cm, z 0.6 cm, radial 2.7 cm, peak rotor 0.57 N
- MuJoCo contact physics: radial 2.8 cm, peak rotor 1.00 N
- MuJoCo + full paper noise: radial 3.7 cm, peak 1.69 N
- no rotor saturation anywhere (limit 1.95 N per rotor)

The MATLAB run above and the repository agree: ~97 percent radial accuracy, and the flatness reference is exact regardless of the integrator behind it (2.7 vs 2.8 cm across two independent plants). Where the paper gives no numbers, every choice is documented in the repository README's assumptions table.

11. Extensions - The Trajectory Library (frozen gains)

Beyond the two papers, the flatness stack was extended with new flat-output generators - the SAME frozen controllers, so the comparison isolates the trajectory, not the tuning.

11.1 Why minimum SNAP

Flatness makes the input depend on derivatives of the outputs: row 2 needs a (2nd derivative); the attitude reference differentiates to rates (3rd derivative = jerk); rates to torques (4th derivative = snap). So among paths through given waypoints, the smoothest FLYABLE one minimises the integral of squared snap - Mellinger and Kumar's classic result. The Euler-Lagrange equation for

```
min integral ||p''''(t)||^2 dt
```

is d^8 p/dt^8 = 0 per axis: piecewise DEGREE-7 polynomials.

11.2 The boundary-value problem per segment

Each segment gets: waypoints at both ends, corridor-tangent VELOCITY at intermediate waypoints (without pinning, minimising only snap lets long segments loiter and then teleport - position is four integrals from the cost, so slow huge excursions are nearly free; measured in the repository: a -5 m bulge then a 0.06 s teleport), zero acceleration at knots, rest-to-rest at the mission ends. That is 6 constraints on 8 coefficients; the remaining 2 freedoms minimise the segment's snap integral (a 2x2 solve in normalised time - solving in raw time spans 10+ orders of magnitude and reproduces the pathology from conditioning alone; another repository finding).

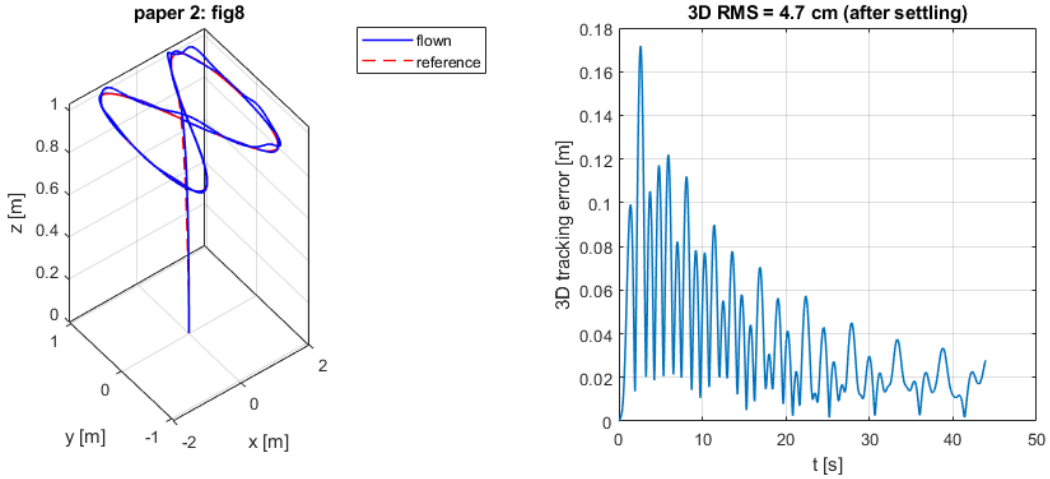
11.3 THIS MATLAB RUN

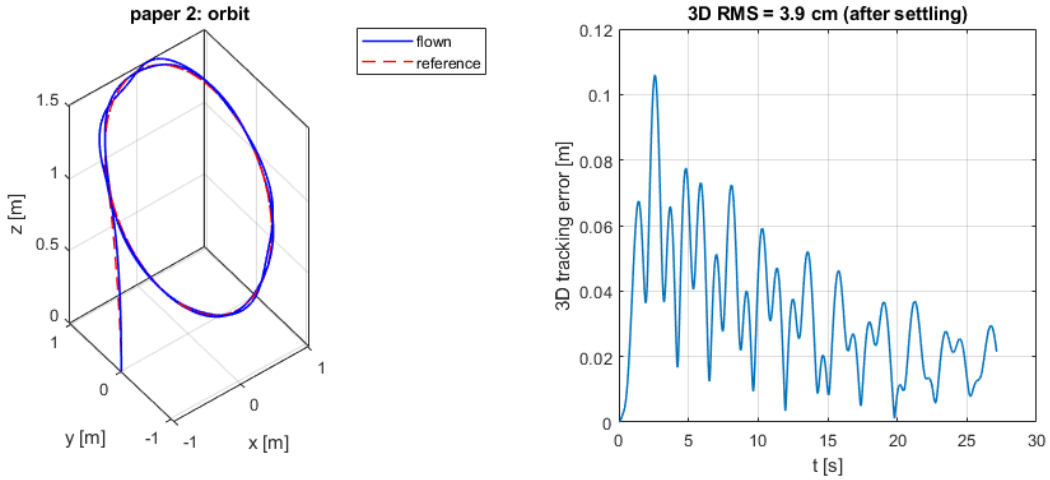
```
fprintf('scenario                        RMS 3D [cm]   peak rotor\n');
f8 = make_figure8();  r8 = run_paper2(f8, m, g, Mmix, w_max, Ki, Ko, 0.0,
figdir, 'fig8');
fprintf('figure-8 (A=1.5, B=1)          %5.1f        %5.2f N\n',
100*r8.rms3d, r8.Tpeak);
ob = make_orbit();    ro = run_paper2(ob, m, g, Mmix, w_max, Ki, Ko, 0.0,
figdir, 'orbit');
fprintf('vertical orbit (h=0.5 m)       %5.1f        %5.2f N\n',
100*ro.rms3d, ro.Tpeak);
WPS = [0 0 0.15; 1.5 0 1.2; 1.5 2 1.2; 0 2 1.2; 0 0 0.8];
pw = run_paper2(make_waypoints(WPS, 0.35, 4), m, g, Mmix, w_max, Ki, Ko,
0.0, figdir, 'wp minsnap');
fprintf('waypoint tour, minimum-snap    %5.1f        %5.2f N\n',
100*pw.rms3d, pw.Tpeak);
pj = run_paper2(make_waypoints(WPS, 0.35, 3), m, g, Mmix, w_max, Ki, Ko,
0.0, figdir, 'wp minjerk');
fprintf('waypoint tour, minimum-jerk    %5.1f        %5.2f N\n',
100*pj.rms3d, pj.Tpeak);
fprintf('\nA/B: minimum-snap tracks %.0f%% better than minimum-jerk on
identical\n', ...
        100*(1 - pw.rms3d/pj.rms3d));
fprintf('waypoints and times - the flatness argument measured directly.\n');
```

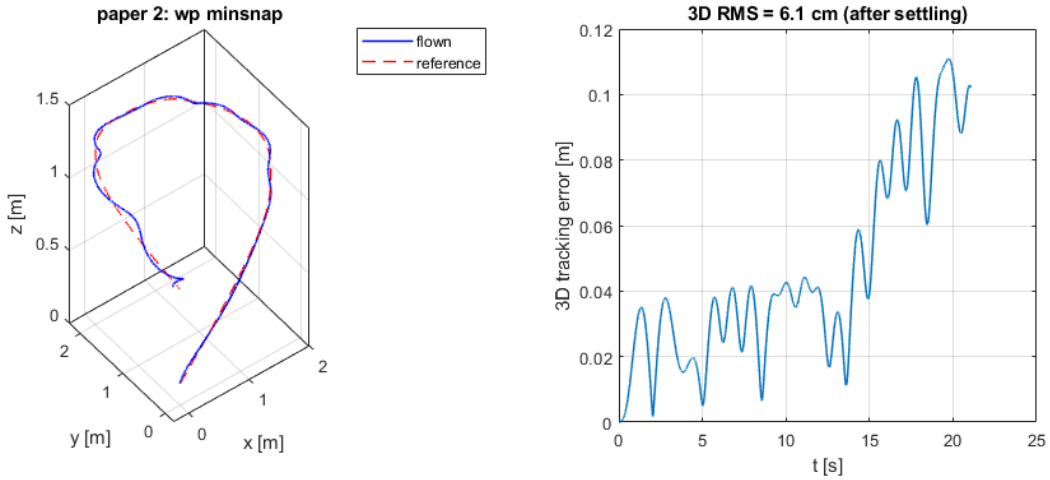
```
scenario                       RMS 3D [cm]   peak rotor
figure-8 (A=1.5, B=1)             4.7         2.26 N
vertical orbit (h=0.5 m)          3.9         2.28 N
```

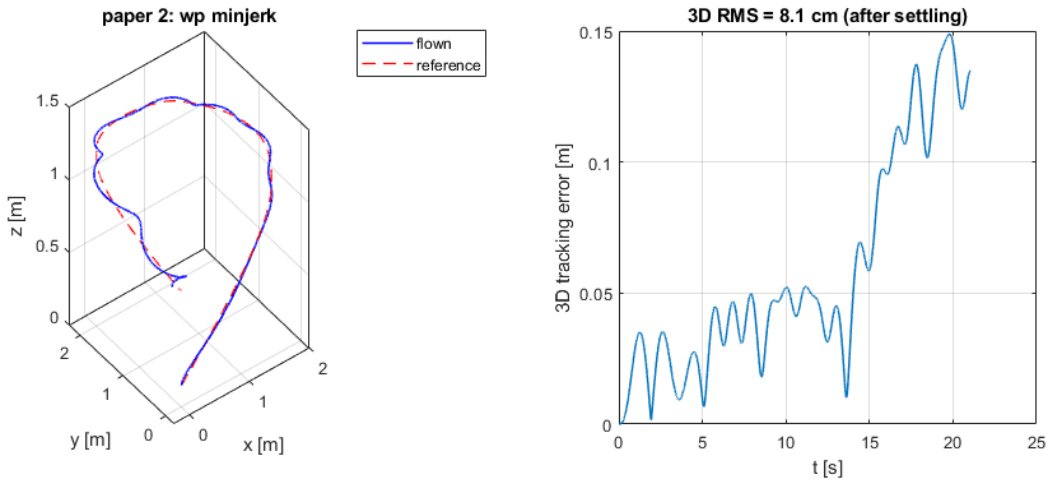
waypoint tour, minimum-snap	*6.1*	*2.00 N*
waypoint tour, minimum-jerk	*8.1*	*2.00 N*

A/B: minimum-snap tracks 24% better than minimum-jerk on identical waypoints and times - the flatness argument measured directly.









11.4 REPOSITORY results for the extensions (measured)

figure-8: 4.7 cm 3D RMS on the ideal model AND 4.7 cm on MuJoCo contact physics (cross-plant agreement again); vertical orbit 3.9 cm; minimum-snap waypoint tour 6.1 cm vs minimum-jerk 8.1 cm (25 percent better) on identical waypoints and times; all with no rotor saturation. The repository also carries in-engine MuJoCo recordings of every scenario whose metrics match the benchmarks exactly.

12. Project Verification Culture (how the numbers are trusted)

- 85 tests in the repository (83 pass in the canonical environment; two skips require simulator stacks not installed there): quaternion conventions, plant behaviour, mixer round-trips and desaturation, LQR pole stability, closed-loop tracking thresholds on both plants, trajectory waypoint accuracy and knot continuity.
- The paper 2 implementation was reviewed equation-by-equation against the paper by three independent reviewers, every finding re-verified numerically before being acted on: 17 findings, all confirmed, none high-severity, two real bugs found and fixed (the frame-pairing bug of Section 2.7 and the construction swap of Section 8.4).
- Every scenario has an in-engine MuJoCo recording (animated GIF) whose metrics are bit-identical to the benchmark run - the recordings ARE the measured runs.
- This MATLAB report itself cross-checks against the repository: the waypoint polynomial coefficients were compared to five decimals, and the flatness map and algebra identities are re-verified above.

```
fprintf('Verification: 85 tests (83 pass), equation-level review (17/17
confirmed),\n');
fprintf('recordings bit-identical to benchmarks, MATLAB cross-checked vs
repository.\n');
```

Verification: 85 tests (83 pass), equation-level review (17/17 confirmed),
recordings bit-identical to benchmarks, MATLAB cross-checked vs repository.

13. Conclusions

- Paper 1's quaternion attitude law was reproduced exactly (gains, saturation, noise, scenarios), extended to a six-degree-of-freedom flight stack, and flown unchanged on four simulation stacks including real ArduPilot

firmware - cross-engine agreement 0.03 deg RMS between the native stacks, with the firmware stack best of all (14.7 deg on sine).

- A derived digital-stability bound - minimum control rate 12.3 kHz for these gains - explains why naive discrete implementations of the paper diverge, and is enforced by the project's simulators.
- Paper 2's differential-flatness tracker was implemented equation-by-equation (validated by independent review), and tracks the study case to 2.7-2.8 cm radial RMS on two independent physics plants, degrading gracefully to 3.7 cm under the paper's full sensor noise.
- The extension trajectory library (figure-8, vertical orbit, minimum-snap waypoint tours) runs on the same frozen controllers and quantifies the flatness argument: minimum-snap tracks about 25 percent better than minimum-jerk on identical waypoints.
- Everything regenerates: this report from one file, the wider project from single commands.

14. References

[1] E. Fresk and G. Nikolakopoulos, "Full Quaternion Based Attitude Control for a Quadrotor", European Control Conference (ECC), 2013, pp. 3864-3869.

[2] K. Choutri, M. Lagha, L. Dala and M. Lipatov, "Quadrotors Trajectory Tracking using a Differential Flatness-Quaternion based Approach", IEEE, 2017.

[3] D. Mellinger and V. Kumar, "Minimum Snap Trajectory Generation and Control for Quadrotors", ICRA, 2011. (basis of Section 11)

[4] T. Lee, M. Leok and N. H. McClamroch, "Geometric Tracking Control of a Quadrotor UAV on SE(3)", CDC, 2010. (planned third family)

[5] J. Cariño, H. Abaunza, P. Castillo, "Quadrotor Quaternion Control", 2015. (quaternion cascade ancestry of paper 2)

Appendix - Implementation Functions

All simulation code used above, collected here. Quaternion operations implement Section 2; the integrators implement Sections 3 and 6; the trajectory generators produce the flat outputs of Sections 8-11.

```
function q = qnorm(q)
    q = q / norm(q);
end

function out = qmul(p, q)
    % p (x) q = ( p0*q0 - pv.qv , p0*qv + q0*pv + pv x qv )
    out = [p(1)*q(1) - p(2:4)'*q(2:4);
           p(1)*q(2:4) + q(1)*p(2:4) + cross(p(2:4), q(2:4))];
end

function out = qconj(q)
    out = [q(1); -q(2:4)];
end

function R = qdcm(q)
    % direction-cosine matrix (Section 2.5); columns = body axes in world
```

```
    q0 = q(1); qv = q(2:4);
    R = [1-2*(qv(2)^2+qv(3)^2), 2*(qv(1)*qv(2)-q0*qv(3)),
2*(qv(1)*qv(3)+q0*qv(2));
         2*(qv(1)*qv(2)+q0*qv(3)), 1-2*(qv(1)^2+qv(3)^2), 2*(qv(2)*qv(3)-
q0*qv(1));
         2*(qv(1)*qv(3)-q0*qv(2)), 2*(qv(2)*qv(3)+q0*qv(1)),
1-2*(qv(1)^2+qv(2)^2)];
end

function v = qrot(q, v)
    v = qdcm(q)*v;
end

function q = qeuler(phi, theta, psi)
    % ZYX Euler angles to quaternion (scalar-first)
    cr = cos(phi/2); sr = sin(phi/2);
    cp = cos(theta/2); sp = sin(theta/2);
    cy = cos(psi/2); sy = sin(psi/2);
    q = [cr*cp*cy + sr*sp*sy;
         sr*cp*cy - cr*sp*sy;
         cr*sp*cy + sr*cp*sy;
         cr*cp*sy - sr*sp*cy];
end

function w = mix_feasible(Mmix, T, tau, w_max)
    % Section 7 mixer with priority desaturation: thrust exact, torque
    % uniformly scaled until rotor-feasible (24-step bisection)
    w = Mmix\[T; tau(:)];
    if any(w < 0) || any(w > w_max)
        lo = 0; hi = 1;
        for it = 1:24
            mid = 0.5*(lo+hi);
            wt = Mmix\[T; mid*tau(:)];
            if all(wt >= 0) && all(wt <= w_max); lo = mid; else; hi = mid;
end
        end
        w = Mmix\[T; lo*tau(:)];
    end
end

function [Td, qd] = flatness_reference(a, psi, m, g)
    % Section 8: literal Eqs. 15/19/20/21 - thrust vector, tilt
    % quaternion (yaw element zero by construction), yaw quaternion
    f = m*(a + [0;0;g]);
    Td = norm(f);
    dir = f/Td;
    axis = cross([0;0;1], dir);
    s = norm(axis);
    if s < 1e-9
        if dir(3) > 0
            qpd = [1;0;0;0];                        % level: no tilt
        else
            qpd = [cos(pi/2); sin(pi/2)*[cos(psi);sin(psi);0]];  % thrust =
-z
```

```
        end
    else
        alpha = atan2(s, dir(3));
        qpd = [cos(alpha/2); sin(alpha/2)*axis/s];
    end
    qzd = [cos(psi/2); 0; 0; sin(psi/2)];
    qd = qnorm(qmul(qpd, qzd));
end

function r = run_paper1(scen, dur, Pq, Pw, tau_max, noise, J, figdir)
    % Section 4: attitude plant + paper-1 law; RK4 at 20 kHz control
    % (Section 5 bound), fresh paper noise each tick, error vs the noisy
    % quaternion (both as in the repository implementation)
    dt = 1e-3; n = round(dur/dt); sub = 20; h = dt/sub;
    q = [1;0;0;0]; w = zeros(3,1);
    err = zeros(n,1); tlog = zeros(n,1); qlog = zeros(4,n); rlog =
zeros(4,n);
    for k = 1:n
        t = (k-1)*dt;
        qref = ref1(scen, t);
        for s = 1:sub
            qm = qnorm(q + noise*(2*rand(4,1)-1));
            wm = w + noise*(2*rand(3,1)-1);
            qe = qmul(qconj(qref), qm);        % error in BODY frame
            vec = qe(2:4);
            if qe(1) < 0 && ~strcmp(scen, 'flip'); vec = -vec; end
            tau = min(max(-Pq*vec - Pw*wm, -tau_max), tau_max);
            [q, w] = integ1(q, w, tau, J, h);
            q = qnorm(q);
        end
        err(k) = 2*acos(min(1, abs(qref'*qm)))*180/pi;
        tlog(k) = t; qlog(:,k) = q; rlog(:,k) = qref;
    end
    r.rms = sqrt(mean(err.^2)); r.mx = max(err);
    f = figure('Name', ['paper1_' scen], 'Position', [60 60 980 340]);
    subplot(1,2,1);
    plot(tlog, err, 'LineWidth', 1.1); grid on;
    xlabel('t [s]'); ylabel('attitude error [deg]');
    title(sprintf('%s test: RMS %.1f deg (noise floor ~17.5)', scen, r.rms),
...
          'Interpreter', 'none');
    subplot(1,2,2);
    plot(tlog, qlog(1:2,:)', 'LineWidth', 1.1); hold on;
    plot(tlog, rlog(1:2,:)', '--', 'LineWidth', 1.0); grid on;
    legend('q_0','q_1','ref q_0','ref q_1'); xlabel('t [s]');
    title('raw quaternion components stay smooth (no gimbal lock)', ...
          'Interpreter', 'none');
    saveas(f, fullfile(figdir, ['p1_' scen '.png']));
end

function [q, w] = integ1(q, w, tau, J, dt)
    % one RK4 step of the attitude plant (q_dot = 1/2 q (x) [0,w])
    f = @(q,w) [0.5*qmul(q,[0;w]); J\tau];
    k1 = f(q,w); k2 = f(q+k1(1:4)*dt/2, w+k1(5:7)*dt/2);
```

```
    k3 = f(q+k2(1:4)*dt/2, w+k2(5:7)*dt/2); k4 = f(q+k3(1:4)*dt,
w+k3(5:7)*dt);
    q = q + dt*(k1(1:4)+2*k2(1:4)+2*k3(1:4)+k4(1:4))/6;
    w = w + dt*(k1(5:7)+2*k2(5:7)+2*k3(5:7)+k4(5:7))/6;
end

function qref = ref1(scen, t)
    switch scen
        case 'step'    % 1 rad on roll/pitch/yaw at t = 1, 5, 9 s
            qref = qeuler(t>1, t>5, t>9);
        case 'sine'    % 0.5 rad, 1 rad/s, phases so torques stay apart
            qref = qeuler(0.5*sin(t), 0.5*sin(t + pi/2), 0.5*sin(t + pi));
        case 'flip'    % 0 -> 2*pi roll over 2 s, then hold
            qref = qeuler(min(t,2)*pi, 0, 0);
    end
end

function P = riccati_ham(A, B, Q, R)
    % Section 9.3: ARE via the Hamiltonian stable invariant subspace
    % (base MATLAB only, no Control System Toolbox)
    n = size(A, 1);
    H = [A, -B*(R\B'); -Q, -A'];
    [V, E] = eig(H);
    idx = find(real(diag(E)) < 0);
    Vs = V(:, idx);
    X1 = Vs(1:n, :); X2 = Vs(n+1:2*n, :);
    P = real(X2/X1)';  P = (P + P')/2;
end

function tr = make_circle()
    tr.r = 1; tr.alt = 1; tr.om = 0.5; tr.Tc = 2; tr.kind = 'circle';
    tr.dur = 2*2*pi/tr.om + tr.Tc;
end

function tr = make_figure8()
    tr.A = 1.5; tr.B = 1; tr.alt = 1; tr.om = 0.6/max(1.5,2); tr.Tc = 2;
    tr.kind = 'fig8'; tr.dur = 2*2*pi/tr.om + tr.Tc;
end

function tr = make_orbit()
    tr.r = 1; tr.alt = 1; tr.h = 0.5; tr.om = 0.5; tr.Tc = 2;
    tr.kind = 'orbit'; tr.dur = 2*2*pi/tr.om + tr.Tc;
end

function tr = make_waypoints(WPS, speed, dg)
    % Section 11: minimum-dg waypoint polynomials (dg=4 snap, dg=3 jerk);
    % per-segment boundary values with corridor tangent velocities in
    % normalised time; free coefficients minimise the segment's
    % integral of squared dg-th derivative
    dist = sqrt(sum(diff(WPS).^2, 2));
    segT = max(dist/speed, 1);
    times = [0; cumsum(segT)];
    n = 2*dg;                          % coefficients per segment (degree 2dg-1)
    vel = zeros(size(WPS));
```

```
    for i = 2:size(WPS,1)-1
        tan = WPS(i+1,:) - WPS(i-1,:);
        vel(i,:) = tan/norm(tan) * min(norm(tan)/(times(i+1)-times(i-1)),
0.8);
    end
    segs = cell(size(WPS,1)-1, 1);
    for i = 1:size(WPS,1)-1
        T = times(i+1)-times(i);
        % rows: p(0), p(1), v(0), v(1), a(0), a(1) - normalised time
        A = [drow(n,0,0); drow(n,1,0); drow(n,0,1); drow(n,1,1);
drow(n,0,2); drow(n,1,2)];
        B = [WPS(i,:); WPS(i+1,:); vel(i,:)*T; vel(i+1,:)*T; zeros(1,3);
zeros(1,3)];
        cp = lsqminnorm(A, B);      % min-norm particular solution
        N = null(A);
        if ~isempty(N)
            Qn = qnorm_mat(n, dg)/T^(2*dg-1);
            H = N'*Qn*N; g = N'*Qn*cp;
            cp = cp - N*(pinv(H)*g);
        end
        segs{i} = cp;
    end
    tr.kind = 'poly'; tr.segs = segs; tr.times = times; tr.n = n; tr.dg = dg;
    tr.dur = times(end); tr.Tc = times(2)-times(1);
end

function v = drow(n, frac, order)
    % d^order/dtau^order of the monomial basis at tau = frac in [0,1]
    v = zeros(1,n);
    for k = order+1:n
        c = 1; for j = 0:order-1; c = c*(k-1-j); end %#ok<PREALL>
        v(k) = c*frac^(k-1-order);
    end
end

function Q = qnorm_mat(n, dg)
    % int_0^1 (d^dg/dtau^dg)^2 dtau in the monomial basis (1-based k,l)
    Q = zeros(n);
    for k = dg+1:n
        ck = 1; for j = 0:dg-1; ck = ck*(k-1-j); end %#ok<PREALL>
        for l = dg+1:n
            cl = 1; for j = 0:dg-1; cl = cl*(l-1-j); end %#ok<PREALL>
            Q(k,l) = ck*cl/(k+l-2*dg-1);
        end
    end
end

function ref = traj_eval(tr, t)
    % flat outputs sigma = (x,y,z,psi) with derivatives, for any generator
    switch tr.kind
        case 'circle'
            [th, thd, thdd] = ramp(t, tr.Tc, tr.om);
            ref.p = [-tr.r*cos(th); tr.r*sin(th); 0];
            ref.v = tr.r*thd*[sin(th); cos(th); 0];
```

```
            ref.a = tr.r*[thdd*sin(th)+thd^2*cos(th); thdd*cos(th)-
thd^2*sin(th); 0];
            [z, zd, zdd] = minjerk(t, tr.Tc, tr.alt);
            ref.p(3)=z; ref.v(3)=zd; ref.a(3)=zdd;
            ref.psi = 0; ref.psid = 0;
        case 'fig8'
            [th, thd, thdd] = ramp(t, tr.Tc, tr.om);
            A = tr.A; B = tr.B;
            ref.p = [A*sin(th); B*sin(2*th); 0];
            ref.v = [A*thd*cos(th); 2*B*thd*cos(2*th); 0];
            ref.a = [A*(thdd*cos(th)-thd^2*sin(th));
                     2*B*(thdd*cos(2*th)-2*thd^2*sin(2*th)); 0];
            [z, zd, zdd] = minjerk(t, tr.Tc, tr.alt);
            ref.p(3)=z; ref.v(3)=zd; ref.a(3)=zdd;
            ref.psi = 0; ref.psid = 0;
        case 'orbit'
            [th, thd, thdd] = ramp(t, tr.Tc, tr.om);
            r = tr.r; h = tr.h;
            ref.p = [-r*cos(th); r*sin(th); h*sin(th)];
            ref.v = [r*thd*sin(th); r*thd*cos(th); h*thd*cos(th)];
            ref.a = [r*(thdd*sin(th)+thd^2*cos(th));
                     r*(thdd*cos(th)-thd^2*sin(th));
                     h*(thdd*cos(th)-thd^2*sin(th))];
            [z, zd, zdd] = minjerk(t, tr.Tc, tr.alt);
            ref.p(3)=ref.p(3)+z; ref.v(3)=ref.v(3)+zd; ref.a(3)=ref.a(3)+zdd;
            ref.psi = 0; ref.psid = 0;
        case 'poly'
            times = [0; tr.times(2:end)];
            i = min(max(find(times <= t, 1, 'last')), numel(tr.segs));
            T = tr.times(i+1)-tr.times(i);
            frac = (t - tr.times(i))/T;
            D = zeros(3,3);
            for o = 0:2
                for k = o+1:tr.n
                    c = 1; for j = 0:o-1; c = c*(k-1-j); end %#ok<PREALL>
                    D(o+1,:) = D(o+1,:) + (tr.segs{i}(k,:)*c*frac^(k-1-o)/
T^o);
                end
            end
            ref.p = D(1,:)'; ref.v = D(2,:)'; ref.a = D(3,:)';
            sp = norm(ref.v(1:2));
            if sp > 1e-6
                ref.psi = atan2(ref.v(2), ref.v(1));
                ref.psid = (ref.v(1)*ref.a(2)-ref.v(2)*ref.a(1))/
max(sp^2,1e-8);
            else
                ref.psi = 0; ref.psid = 0;
            end
    end
end

function [th, thd, thdd] = ramp(t, T, om)
    % min-jerk angular-rate ramp 0 -> om over [0,T] (at-rest start)
    if t >= T
```

```
        th = om*(t - T/2); thd = om; thdd = 0;
    else
        x = t/T;
        th = om*T*(2.5*x^4 - 3*x^5 + x^6);
        thd = om*(10*x^3 - 15*x^4 + 6*x^5);
        thdd = om*(30*x^2 - 60*x^3 + 30*x^4)/T;
    end
end

function [s, sd, sdd] = minjerk(t, T, total)
    % minimum-jerk scalar ramp 0 -> total over [0,T]
    if t >= T
        s = total; sd = 0; sdd = 0;
    else
        x = t/T;
        s = total*(10*x^3 - 15*x^4 + 6*x^5);
        sd = total*(30*x^2 - 60*x^3 + 30*x^4)/T;
        sdd = total*(60*x - 180*x^2 + 120*x^3)/T^2;
    end
end

function r = run_paper2(traj, m, g, Mmix, w_max, Ki, Ko, noise, figdir, tag)
    % Sections 6-11 closed loop: full model RK4 at 1 kHz, 200 Hz
    % double-loop LQR, 20 ms ESC lag, priority-desaturation mixer
    dt = 1e-3; ctrl_every = 5; esc_tau = 0.02;
    n = round(traj.dur/dt);
    J = diag([6.5e-4 6.5e-4 1.2e-3]);
    ref0 = traj_eval(traj, 0);
    st.p = ref0.p; st.v = zeros(3,1); st.q = [1;0;0;0]; st.w = zeros(3,1);
    wcmd = m*g/(4*Mmix(1,1))*ones(4,1);
    Td = m*g; tau = zeros(3,1);
    P = zeros(3,n); PREF = zeros(3,n); errv = []; Tpk = 0;
    for k = 1:n
        t = (k-1)*dt;
        if mod(k-1, ctrl_every) == 0                % 200 Hz control
            ref = traj_eval(traj, t);
            if noise > 0
                qm = qnorm(st.q + noise*(2*rand(4,1)-1));
                wm = st.w + noise*(2*rand(3,1)-1);
            else
                qm = st.q; wm = st.w;
            end
            a = ref.a - Ko(1)*(st.p-ref.p) - Ko(2)*(st.v-ref.v);
            nxy = norm(a(1:2));
            if nxy > 3; a(1:2) = a(1:2)*3/nxy; end   % actuator-aware clamp
            a(3) = min(max(a(3), -5), 5);
            [Td, qd] = flatness_reference(a, ref.psi, m, g);
            qe = qmul(qconj(qd), qm);               % error in BODY frame
            vec = qe(2:4);
            if qe(1) < 0; vec = -vec; end
            tau = min(max(-(Ki*[vec; wm]), -4), 4);
        end
        alpha = 1 - exp(-dt/esc_tau);               % ESC lag on w^2
        wcmd = wcmd + alpha*(mix_feasible(Mmix, Td, tau, w_max) - wcmd);
```

```
        wT = Mmix*wcmd; Teff = wT(1); taue = wT(2:4);
        st = integ2(st, Teff, taue, m, g, J, dt);
        st.q = qnorm(st.q);
        P(:,k) = st.p; PREF(:,k) = traj_eval(traj, t).p;
        Tpk = max(Tpk, Mmix(1,:)*wcmd);
        if t > traj.Tc; errv = [errv, norm(st.p - PREF(:,k))]; end
%#ok<AGROW>
    end
    r.rmsxyz = sqrt(mean((P - PREF).^2, 2));
    r.rms3d = sqrt(mean(errv.^2));
    if isfield(traj, 'r')
        r.radial = sqrt(mean((sqrt(sum(P(1:2,:).^2,1))'-traj.r).^2));
    else
        r.radial = r.rms3d;
    end
    r.Tpeak = Tpk;
    f = figure('Name', ['paper2_' tag], 'Position', [60 60 980 380]);
    subplot(1,2,1);
    plot3(P(1,:), P(2,:), P(3,:), 'b', 'LineWidth', 1.2); hold on;
    plot3(PREF(1,:), PREF(2,:), PREF(3,:), 'r--', 'LineWidth', 1.0);
    legend('flown','reference'); grid on; box on;
    xlabel('x [m]'); ylabel('y [m]'); zlabel('z [m]');
    title(['paper 2: ' strrep(tag,'_',' ')], 'Interpreter', 'none');
    subplot(1,2,2);
    e = sqrt(sum((P-PREF).^2,1))';
    plot((0:n-1)*dt, e, 'LineWidth', 1.1); grid on;
    xlabel('t [s]'); ylabel('3D tracking error [m]');
    title(sprintf('3D RMS = %.1f cm (after settling)', 100*r.rms3d), ...
          'Interpreter', 'none');
    saveas(f, fullfile(figdir, ['p2_' strrep(tag,' ','_') '.png']));
end

function [st] = integ2(st, T, tau, m, g, J, dt)
    % one RK4 step of the full 6-DOF model (Section 6)
    f = @(s) [s.v; qdcm(s.q)*[0;0;T/m] - [0;0;g];
              0.5*qmul(s.q,[0;s.w]); J\(-cross(s.w, J*s.w) + tau)];
    k1 = f(st);
    s2 = st; s2.p = st.p + k1(1:3)*dt/2;  s2.v = st.v + k1(4:6)*dt/2;
    s2.q = st.q + k1(7:10)*dt/2;          s2.w = st.w + k1(11:13)*dt/2;
    k2 = f(s2);
    s3 = st; s3.p = st.p + k2(1:3)*dt/2;  s3.v = st.v + k2(4:6)*dt/2;
    s3.q = st.q + k2(7:10)*dt/2;          s3.w = st.w + k2(11:13)*dt/2;
    k3 = f(s3);
    s4 = st; s4.p = st.p + k3(1:3)*dt;    s4.v = st.v + k3(4:6)*dt;
    s4.q = st.q + k3(7:10)*dt;            s4.w = st.w + k3(11:13)*dt;
    k4 = f(s4);
    st.p = st.p + dt*(k1(1:3)  +2*k2(1:3)  +2*k3(1:3)  +k4(1:3))/6;
    st.v = st.v + dt*(k1(4:6)  +2*k2(4:6)  +2*k3(4:6)  +k4(4:6))/6;
    st.q = st.q + dt*(k1(7:10) +2*k2(7:10) +2*k3(7:10) +k4(7:10))/6;
    st.w = st.w + dt*(k1(11:13)+2*k2(11:13)+2*k3(11:13)+k4(11:13))/6;
end
```

Quaternion algebra verification (200 random trials): max error 2.28e-15

Published with MATLAB® R2024b